Rust: Engineering Safety

Software Engineering Radio Podcast

Episode 690 - Florian Gilcher on Rust for Safety-Critical Systems

Fernando Cachón Alonso
Alejandro de San Claudio Mesa
Bilal Yazicioğlu

The speaker: Florian Gilcher

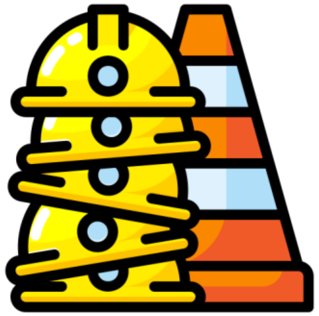
- Managing Director of Ferrous Systems and co-founder of the Rust Foundation

Defining the stakes

- Mission-Critical

- Systems where failure results in fiscal or operational damage
- (e.g., cybersecurity breach).

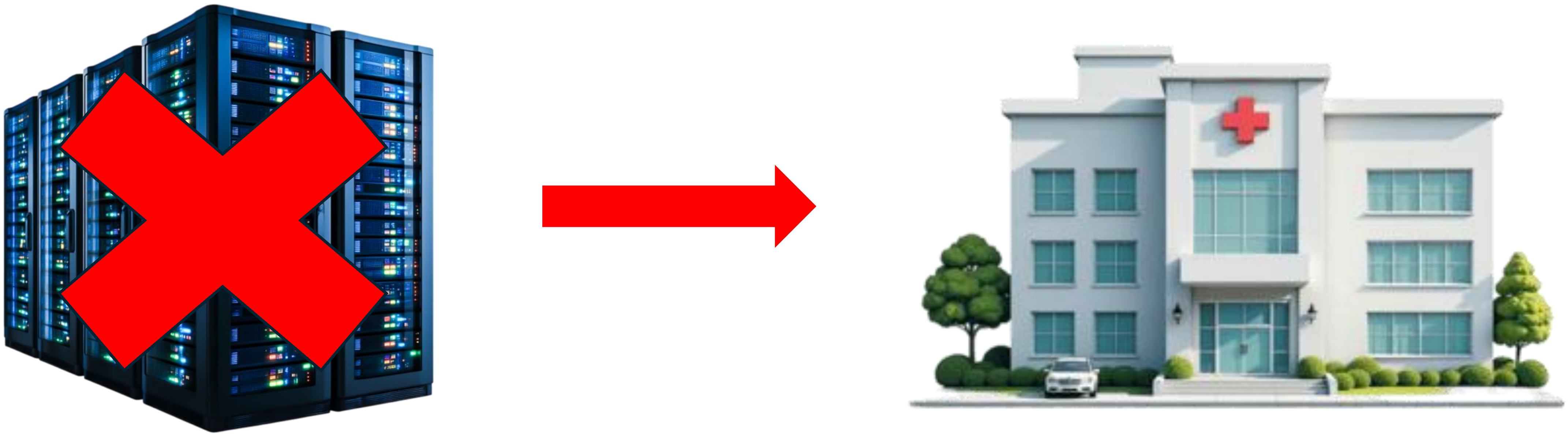
- Safety-Critical



- Systems where failure puts human lives at stake or causes physical harm
- (e.g., automotive, avionics, medical devices failures)

Don’t some mission critical failures lead into safety critical failures?

- Mission-Critical
- Safety-Critical



The Regulatory Landscape: Safety Standards and Practices

INDUSTRY STANDARDS: **ISO 26262** (AUTOMOTIVE), **IEC 61508** (INDUSTRIAL), AND **DO-178** (AVIONICS).

STANDARDS OFTEN **"HIGHLY RECOMMEND"** USING **STATICALLY TYPED PROGRAMMING LANGUAGES**

THE VALIDATION PRINCIPLE: A CRITICAL PRACTICE IS THE INTERPLAY BETWEEN THE IMPLEMENTER AND AN INDEPENDENT VALIDATOR TO ENSURE SOFTWARE QUALITY IS NOT ASSESSED BY THOSE WHO WROTE IT.

The Regulatory Landscape: Rust as a helper for safety standards

- Static Typing
- Restricted pointer usage
- Memory and thread safety
- Predictable runtime
- Formal specification

Why Rust for High-Assurance Software?

Core Mechanics

Runtime Performance

Standards Alignment

Tradicional(C/C++)	Rust
Manual memory management	Compiler-enforced memory safety
Concurrency is difficult	Built-in concurrency safety
Bugs found at runtime	Most memory bugs at compile time
Requires external tools	Safety is a core feature

Ownership & Borrowing

Rust's core concepts ensuring it's clear who can use memory and for how long. The compiler tracks this.

No runtime, no garbage collector. Just compile-time checks, making Rust as performant as C.

If the Rust compiler is high quality, why need a 'qualified' version?

"I **don't** want to be in a car where someone said 'The brakes feels good.'

The brakes **have to work**.

Qualification Adds

- FORMAL SPEC: A WRITTEN DOC DEFINING HOW THE LANGUAGE SHOULD BEHAVE.
- TEST COVERAGE: EVIDENCE THAT EVERY FEATURE IS THOROUGHLY TESTED.
- LONG-TERM SUPPORT: PATCHES AND SUPPORT FOR SPECIFIC COMPILER VERSIONS.
- GUIDANCE & DOCS: CLEAR ADVICE ON WHICH FEATURES TO USE FOR MAX SAFETY.

The Real World

There is lots of companies that are using Rust or planning to turn all their codebase to the Rust

Google: Scaling Productivity

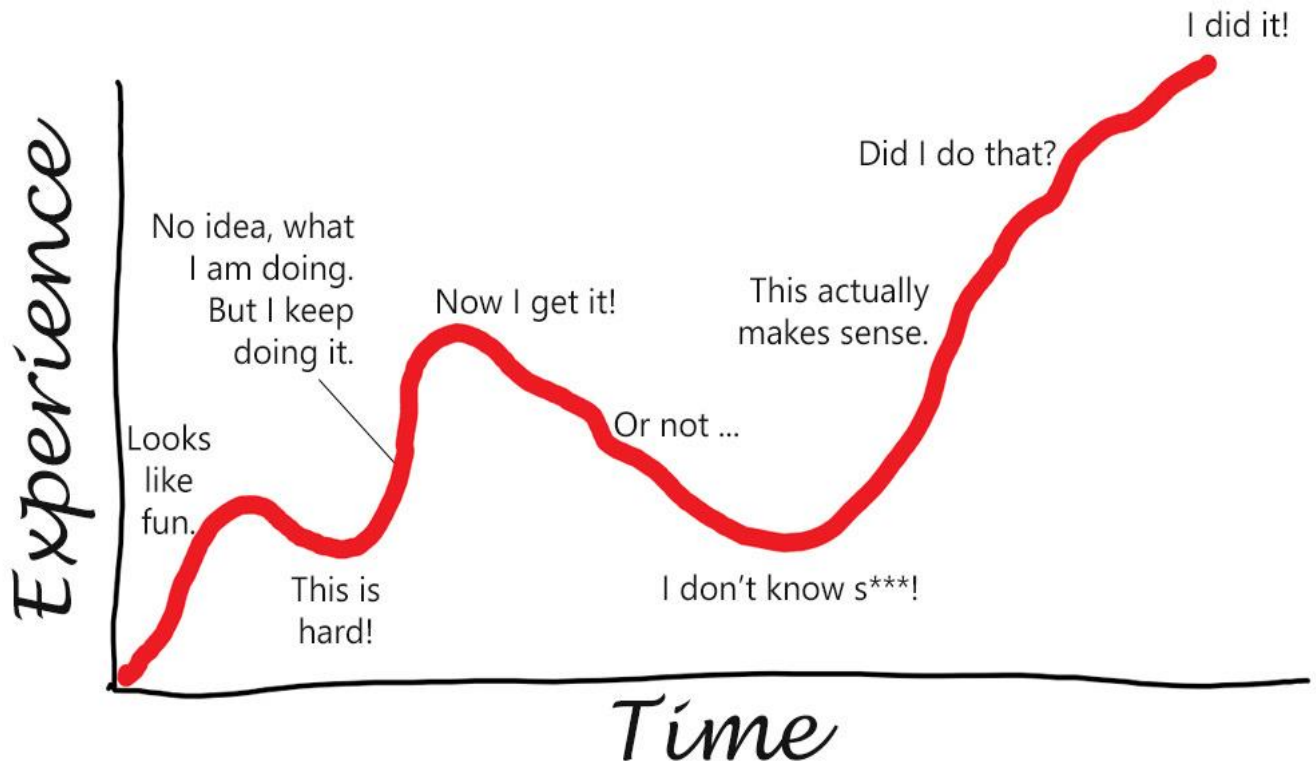
- **2x Development Velocity: Teams reported a doubling in productivity when switching to Rust for specific services.**
- **Streamlined Code Reviews: Significantly less time spent in the review phase because the compiler acts as the first line of defense.**
- **Early Bug Detection: Unlike C++, where memory bugs often surface weeks later in production, Rust catches them instantly during compilation.**

Volvo: Transforming Automotive Software

- **Massive Efficiency Gains: Reported a 2 to 4x productivity boost in embedded systems development.**
- **Safety-Critical Reliability: Leveraging memory safety to build more robust vehicle software with fewer crashes and higher uptime.**

Tech Giants That Switching To the Rust
Microsoft: They are planning to finish it before 2030
Cloudflare: Planning to go rust because of C memory security
Discord: They are switching some services to Rust
1Password: They writed the whole codebase in Rust again

Rust Learning Curve:



Rust has hard looking Learning curve But It has great community to support your improvance.

Rust is one of the bests in complier logs. You can learn so many things from your mistakes on Rust Complier.

Rust: Engineering Safety

Software Engineering Radio Podcast

Episode 690 - Florian Gilcher on Rust for Safety-Critical Systems

Fernando Cachón Alonso
Alejandro de San Claudio Mesa
Bilal Yazicioğlu